Strings and Regular Expressions

#	Scenario (Short)	Level	Target Roles
1	Normalize irregular spacing to single space	L1	Backend Core, Full Stack
2	Case-insensitive keyword search	L1	Backend Core, Search Systems
3	Extract yyyy-mm-dd or mm/dd/yyyy	L2	Backend Core, Data Processing
4	Collapse all whitespace (tabs/newlines)	L1	Backend Core
5	Extract hashtags (#Java, #Java_Programming)	L1	Backend Core, Social Media
6	Clean messy input (trim, uppercase, format)	L1	Backend Core
7	Efficient string concatenation (logs/files)	L1	Backend Core
8	Compare strings ignoring case	L1	Backend Core
9	Validate email address	L2	Backend Core, Web Apps
10	Find all keyword matches (case-insensitive)	L2	Backend Core
11	Count word occurrences	L2	Backend Core, Data Processing
12	Extract words starting with “J/j”	L2	Backend Core
13	Reverse order of words in sentence	L1	Backend Core
14	Detect if string contains digits	L1	Backend Core
15	Extract all email addresses	L2	Backend Core
16	Extract paragraphs containing keyword	L2	Backend Core
17	Extract URLs from text	L2	Backend Core, Web Apps
18	Validate SKU like AB-1234-5678	L2	Backend Core, E-commerce
19	Extract text inside square brackets	L2	Backend Core

20	Search multiple keywords (OR)	bar	baz)`
21	Extract hashtags from tweet	L1	Backend Core, Social Media
22	Filter only ERROR lines from logs	L2	Backend Core, DevOps
23	Normalize varied phone formats	L2	Backend Core
24	Parse log lines for severity + message	INFO	WARN): (.*)`)
25	Escape regex metacharacters safely	L3	Backend Core, Search Systems
26	Validate multiple date formats	L3	Backend Core
27	Count pattern occurrences efficiently	L3	Backend Core, Data Engineering
28	Extract emails inside text	L1	Backend Core
29	Validate SKU XXX-YYYY-ZZZZ	L2	Backend Core, Retail Systems
30	Stream CSV and filter valid emails	L3	Backend Core, Data Processing
31	Normalize product names	L1	Backend Core, Retail Apps
32	Extract yyyy-MM-dd HH:mm:ss timestamps	L2	Backend Core, Logging Systems
33	Extract text inside quotes	L2	Backend Core
34	Validate URL starting with http/https	L2	Backend Core
35	Apply multiple find-replace terms	L3	Backend Core
36	Check if string starts with word (ignore case)	L2	Backend Core
37	Validate international phone numbers	L3	Backend Core
38	Extract all words of N characters	L2	Backend Core
39	Check filename ends with .txt (ignore case)	L1	Backend Core

40	Split simple CSV line into fields	L1	Backend Core
41	Distinguish null, empty, blank	L1	Backend Core
42	Extract domain from URL	L1	Backend Core
43	Why "Hello"==... false for new String	L1	Backend Core
44	Sort list of strings alphabetically	L1	Backend Core
45	Sort ignoring case	L1	Backend Core
46	Build large SQL queries efficiently	L2	Backend Core, Data Systems
47	Why StringBuffer in multi-threaded logging	L2	Backend Core, Logging Systems
48	Reverse a string efficiently	L1	Backend Core
49	Format invoice number INV-0001	L1	Backend Core
50	Format floating number to 2 decimals	L1	Backend Core
51	Print timestamp with printf/format	L1	Backend Core
52	Extract IPv4 from logs	L2	Backend Core, Networking
53	Validate currency like $123.45	L2	Backend Core, Fintech
54	Extract HTML tags + attributes	L3	Backend Core, Web Systems
55	Check only alphabetic characters	L1	Backend Core
56	Validate password strength	L3	Backend Core, Security
57	Detect + validate multiple date formats	L3	Backend Core
58	Escape search terms & highlight	L3	Backend Core, Search Systems
59	Explain String pooling & intern()	L2	Backend Core, Java Fundamentals

60	Localized messages with currency format	L3	Backend Core, Global Products
61	Extract domain from email	L2	Backend Core, Communication Systems

Scenario 1: Normalizing irregular spacing in user input

In many real-world systems, like an online job portal or e-commerce form, users may type text with inconsistent spacing. For example, they might write "Senior Software Engineer" with multiple spaces. We need to normalize this to a clean "Senior Software Engineer".

Answer:

The simplest way in Java is to use a regular expression. The pattern \s+ matches one or more whitespace characters (spaces, tabs, or newlines). Replacing them with a single space ensures uniform spacing.

Code:

```
public class NormalizeSpacing {
    public static void main(String[] args) {
        String rawText = "Java    programming   is    awesome!";
        String cleanedText = rawText.replaceAll("\\s+", " ");
        System.out.println(cleanedText);  // Output: Java programming is awesome!
    }
}
```

Reasoning to interviewer:

- Using regex with \s+ collapses any run of whitespace into just one space.
- This approach is flexible since it handles tabs and newlines, not just multiple spaces.
- It keeps the solution short and avoids writing loops or manual trimming.

Future scope:

- Add a .trim() to remove leading or trailing spaces if user input might start or end with blanks.
- Integrate it into input validation pipelines so all text is standardized before saving into a database.
- For multilingual systems, extend regex patterns to normalize other spacing characters like non-breaking spaces.

Scenario 2: Case-insensitive keyword search

Imagine a blog search feature where users type "java" but the article might contain "Java" or "JAVA". We want the search to match regardless of case.

Answer:

In Java, the Pattern.CASE_INSENSITIVE flag makes regex match text in a case-agnostic way. This means "java", "JAVA", or "JaVa" are all considered the same during search.

Code:

```
import java.util.regex.*;

public class CaseInsensitiveSearch {
    public static void main(String[] args) {
        String content = "Learning Java is fun! JAVA programming rocks.";
        String searchTerm = "java";

        Pattern pattern = Pattern.compile(searchTerm,
Pattern.CASE_INSENSITIVE);
        Matcher matcher = pattern.matcher(content);

        while (matcher.find()) {
            System.out.println("Keyword found: " + matcher.group());
        }
    }
}
```

Reasoning to interviewer:

- Using Pattern.CASE_INSENSITIVE makes the regex engine normalize case internally, so we don’t need to lowercase both strings manually.
- This allows flexible searching, especially when users might type in any case.
- The Matcher gives us not only the presence of the word but also its exact occurrences, which can be used for highlighting in UI.

Future scope:

- Combine with Pattern.UNICODE_CASE for internationalized case-insensitive searches.
- Extend logic to highlight matches in the text.
- Use groups in regex for more advanced keyword extraction, like searching for multiple keywords at once.

Scenario 3: Extracting dates from text with multiple formats

In real applications like email parsing or log analysis, we often encounter dates in different formats. For example, "2021-12-25" in an ISO-like format, or "12/25/2021" in a U.S. style. We want a regex that can handle both yyyy-mm-dd and mm/dd/yyyy.

Answer:

I'll write a regex that alternates between the two formats.

- \d{4}[-/]\d{2}[-/]\d{2} → matches yyyy-mm-dd or yyyy/mm/dd.
- \d{2}[-/]\d{2}[-/]\d{4} → matches mm-dd-yyyy or mm/dd/yyyy.
  I'll combine them with a non-capturing group (?: … | …) so either pattern is matched.

Code:

```
import java.util.regex.*;

public class DateExtractor {
    public static void main(String[] args) {
        String textWithDates = "Meetings are on 2021-12-25 and 12/25/2021.";
        String datePattern = "(?:\\d{4}[-/]\\d{2}[-/]\\d{2}|\\d{2}[-/]\\d{2}[-/]\\d{4})";

        Pattern pattern = Pattern.compile(datePattern);
        Matcher matcher = pattern.matcher(textWithDates);

        while (matcher.find()) {
            System.out.println("Date found: " + matcher.group());
        }
    }
}
```

Reasoning to interviewer:

- The regex covers both common styles with just one expression.
- I used a non-capturing group to avoid unnecessary group indices since I only care about the full match.
- The delimiter is written as [-/] so it matches either a dash or a slash.
- This keeps the solution simple and extensible—if new formats come up, I can add them to the regex.

Future scope:

- Integrate with LocalDate.parse to validate and convert extracted strings into proper date objects.
- Extend regex to support European formats like dd-mm-yyyy.
- Use named groups if I want to extract month/day/year parts separately.

Scenario 4: Normalizing whitespace in user input

This is a common input-cleaning problem. Suppose a blog comment field contains "Normalize these spaces.". We want to collapse multiple spaces (or tabs, or line breaks) into a single space.

Answer:

The regex \s+ matches any sequence of whitespace characters. Replacing it with a single space ensures consistent spacing.

Code:

```
public class NormalizeSpaces {
    public static void main(String[] args) {
        String text = "Normalize    these   spaces.";
        String result = text.replaceAll("\\s+", " ");
        System.out.println(result);  // Output: Normalize these spaces.
    }
}
```

Reasoning to interviewer:

- Using regex here avoids writing a loop to check characters one by one.
- \s is more powerful than just a space—it also catches tabs and newlines.
- This is a lightweight operation and works directly on user input strings.

Future scope:

- Add a .trim() after replacement to remove leading/trailing spaces.
- Use it as part of a general input sanitization utility (for usernames, comments, etc.).
- For performance-critical text processing, pre-compile the regex into a Pattern.

Scenario 5: Extracting hashtags from text

Think of a social media app where users post captions with hashtags like #Java, #regex, or #Java_Programming. The requirement is to scan user text and pull out all hashtags for indexing or analytics.

Answer:

The regex #\w+ does the job.

- # looks for the hash symbol.
- \w+ matches one or more word characters (letters, digits, underscores).

  This way, hashtags such as #Java, #123, or #Java_Programming are all matched.

Code:

```
import java.util.regex.*;

public class HashtagFinder {
    public static void main(String[] args) {
        String sampleText = "Follow #Java, #regex, and #Java_Programming!";
        String hashtagRegex = "#\\w+";

        Pattern pattern = Pattern.compile(hashtagRegex);
        Matcher matcher = pattern.matcher(sampleText);

        while (matcher.find()) {
            System.out.println("Hashtag found: " + matcher.group());
        }
    }
}
```

Reasoning to interviewer:

- \w in Java regex covers [A-Za-z0-9_], which fits hashtag rules.
- The Matcher.find() loop ensures we capture all occurrences, not just the first.
- This approach is simple and efficient for extracting keywords in real-time from text streams.

Future scope:

- Extend the regex to handle Unicode hashtags (like #□□□□□ or #日本語) by using \p{L} instead of \w.

- Store hashtags in a Set to avoid duplicates before persisting in a database.
- Add analytics, such as counting hashtag frequency for trending topics.

Scenario 6: Formatting user input text

Imagine a banking chatbot where a user types something messy like " transfer 1000 to savings ". Before displaying or logging, we want to clean the text—remove unnecessary spaces, convert it to uppercase, and wrap it in a standard sentence format.

Answer:

This can be done by chaining built-in String methods:

- trim() to remove leading and trailing whitespace.
- toUpperCase() to normalize casing.
- String.format() to embed it into a standard output message.

Code:

```
public class InputFormatter {
    public static void main(String[] args) {
        String userInput = "  hello world of java!  ";

        userInput = userInput.trim();              // Step 1: remove leading/trailing spaces
        userInput = userInput.toUpperCase();       // Step 2: convert to uppercase
        String result = String.format("Output: %s", userInput); // Step 3: format

        System.out.println(result);  // Output: Output: HELLO WORLD OF JAVA!
    }
}
```

Reasoning to interviewer:

- Each step uses a dedicated String method, keeping the logic readable.
- This is a safe, immutable approach since String methods always return new objects without mutating the original.
- String.format() makes the final message flexible—you can easily embed it into templates for logging, UI, or reports.

Future scope:

- Localize case conversions with toUpperCase(Locale.XX) for international applications.

- Extend the utility into a reusable Formatter class that supports additional transformations (like replacing multiple spaces inside the sentence with one).
- Integrate with input validation libraries for a complete preprocessing pipeline.

Scenario 7: Efficient string concatenation for large text files

Suppose you're building a log processing tool where text lines are continuously appended. If you use plain String with the + operator, every concatenation creates a new object because String is immutable. That quickly becomes inefficient when dealing with thousands of lines.

Answer:

The best class here is StringBuilder.

- StringBuilder is mutable, so append operations don't create new objects.
- It is faster than StringBuffer because it's not synchronized, which is fine in single-threaded environments.
- StringBuffer is only preferable if multiple threads will append concurrently to the same buffer.
- Plain String concatenation should be avoided for frequent modifications because it leads to too many temporary objects.

Code:

```
public class StringConcatenationDemo {
    public static void main(String[] args) {
        StringBuilder builder = new StringBuilder();
        builder.append("Hello ");
        builder.append("World!");
        builder.append(" Let's process logs efficiently.");

        System.out.println(builder.toString());
    }
}
```

Reasoning to interviewer:

- StringBuilder maintains a resizable character array internally, so appending is an amortized O(1) operation.
- This makes it ideal when strings are frequently modified, such as log aggregation or building large SQL queries dynamically.
- In contrast, StringBuffer adds the overhead of synchronization which is unnecessary in single-threaded processing.

Future scope:

- If concurrency is later introduced, switch to StringBuffer or wrap the builder in custom synchronization.
- For extremely large text (gigabytes), consider streaming approaches with BufferedWriter or FileChannel instead of keeping everything in memory.

Scenario 8: Case-insensitive string comparison

Imagine a login screen where a user enters their email in lowercase, but the database stores it in uppercase. The system should still recognize them as equal.

Answer:

The method equalsIgnoreCase() from the String class is designed for this.

- It compares two strings character by character.
- It ignores case differences, so "Java", "JAVA", and "java" are treated as equal.

Code:

```
public class CaseInsensitiveComparison {
    public static void main(String[] args) {
        String first = "Java";
        String second = "JAVA";

        boolean match = first.equalsIgnoreCase(second);
        System.out.println(match);  // Output: true
    }
}
```

Reasoning to interviewer:

- This method avoids the need to manually convert both strings to lowercase or uppercase before comparison.
- It’s cleaner, reduces errors, and is optimized for case-insensitive checks.
- Useful in user-facing applications where input format is unpredictable.

Future scope:

- For locale-sensitive data (like Turkish dotted vs dotless “i”), use toLowerCase(Locale) before comparison to avoid unexpected mismatches.
- When partial matches or pattern-based searches are needed, combine with regex (Pattern.CASE_INSENSITIVE).

Scenario 9: Validating email addresses with regex

In most applications—banking, e-commerce, or even signup pages—we don't want to store invalid emails in the database. A quick format check with regex helps ensure the input looks like a valid email before we proceed.

Answer:

A practical regex for email validation is:

^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$

- The first part ^[a-zA-Z0-9._%+-]+ allows letters, digits, and common symbols before the @.
- Then @[a-zA-Z0-9.-]+ checks the domain part.
- Finally, \.[a-zA-Z]{2,}$ ensures the top-level domain has at least two letters like .com, .org, .in.

Code:

```
import java.util.regex.*;

public class EmailCheck {
    public static void main(String[] args) {
        String inputEmail = "user@example.com";
        String emailPattern = "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$";

        Pattern pattern = Pattern.compile(emailPattern);
        Matcher matcher = pattern.matcher(inputEmail);

        if (matcher.matches()) {
            System.out.println("Email is valid.");
        } else {
            System.out.println("Email is invalid.");
        }
    }
}
```

Reasoning to interviewer:

- This regex covers the common cases and prevents obvious errors like missing @ or .com.
- Using Pattern.matcher(...).matches() checks the whole string, not just a substring, so partial matches won't pass.

- It’s lightweight and runs fast, making it suitable for form validation.

Future scope:

- This regex won’t catch every possible valid email (like those with quoted strings), but it’s good enough for most business apps.
- For production-grade validation, consider javax.mail.internet.InternetAddress which follows RFC rules.
- Add domain verification via DNS MX record lookup if we need to ensure the email is deliverable.

Scenario 10: Case-insensitive keyword search in text

Think of a blogging platform where a user searches for “java”, but the content might contain “Java”, “JAVA”, or mixed cases. We need to return all matches regardless of casing.

Answer:

Java’s Pattern.CASE_INSENSITIVE flag makes regex matching ignore letter case. That way, "java", "Java", and "JAVA" are treated as the same.

Code:

```
import java.util.regex.*;

public class KeywordSearch {
    public static void main(String[] args) {
        String content = "Java is great, java is fun!";
        String searchWord = "java";

        Pattern pattern = Pattern.compile(searchWord,
Pattern.CASE_INSENSITIVE);
        Matcher matcher = pattern.matcher(content);

        while (matcher.find()) {
            System.out.println("Found keyword: " + matcher.group());
        }
    }
}
```

Reasoning to interviewer:

- Pattern.CASE_INSENSITIVE avoids manually lowercasing both strings before comparison.
- matcher.find() scans through the entire text and captures each occurrence, which is useful for highlighting or counting keyword frequency.
- This makes the search logic clean and adaptable to any user-entered keyword.

Future scope:

- Combine with Pattern.UNICODE_CASE for internationalized searches where casing rules differ.
- Extend the logic to highlight matches in the UI or return their positions.

- For large datasets, integrate with indexing tools (like Lucene) but still rely on this approach for quick string checks.

Scenario 11: Counting occurrences of a word in text (case-insensitive)

Suppose you're analyzing customer reviews on an e-commerce site and want to count how many times the word "refund" appears. The reviews might contain "Refund", "REFUND", or "refund". A case-insensitive approach ensures all of them are counted.

Answer:

I'll use Java's regex API with Pattern.CASE_INSENSITIVE. Compiling the target word with this flag ensures matches are case-agnostic. Then, I'll loop through the matches using matcher.find() and keep a counter.

Code:

```
import java.util.regex.*;

public class WordCounter {
    public static void main(String[] args) {
        String inputText = "Java is great. java is fun. Learning Java is awesome!";
        String targetWord = "java";

        Pattern pattern = Pattern.compile(targetWord, Pattern.CASE_INSENSITIVE);
        Matcher matcher = pattern.matcher(inputText);

        int occurrences = 0;
        while (matcher.find()) {
            occurrences++;
        }

        System.out.println("The word '" + targetWord + "' appears " + occurrences + "
times.");
    }
}
```

Reasoning to interviewer:

- Using regex here is flexible and avoids manually converting text to lowercase and splitting it.
- The Pattern.CASE_INSENSITIVE flag makes the logic clean and reliable for user input.
- Each call to matcher.find() jumps to the next occurrence, making it straightforward to count appearances.

Future scope:

- For whole-word matching, wrap the target in \b boundaries (e.g., \bjava\b) so substrings like “javascript” aren’t counted.
- For very large files, stream the input line by line to avoid loading the entire text into memory.
- Extend this to count frequencies of multiple words by maintaining a Map<String, Integer>.

Scenario 12: Extracting words starting with “J” (case-insensitive)

Imagine you’re building a tag extractor for a blog, and you want to pull out all words that start with “J” like “Java”, “JavaScript”, or “JSON”, regardless of case.

Answer:

A regex with word boundaries works well here: \bJ\w*\b.

- \b ensures we only match at word boundaries, so partial matches inside words aren’t included.
- J\w* matches “J” followed by zero or more word characters.
- Compiling with Pattern.CASE_INSENSITIVE allows capturing both uppercase “J” and lowercase “j”.

Code:

```
import java.util.regex.*;

public class WordsExtractor {
    public static void main(String[] args) {
        String sampleText = "Java is great. JavaScript is amazing! JSON is widely
used.";
        String patternString = "\\bJ\\w*\\b";  // Matches words starting with "J"

        Pattern pattern = Pattern.compile(patternString,
Pattern.CASE_INSENSITIVE);
        Matcher matcher = pattern.matcher(sampleText);

        while (matcher.find()) {
            System.out.println("Word found: " + matcher.group());
        }
    }
}
```

Reasoning to interviewer:

- \b ensures we only capture complete words like “Java”, not substrings in “ObjectJava”.
- \w* keeps the pattern flexible—it can handle “J” alone or longer words like “JavaScript”.
- The CASE_INSENSITIVE flag makes it robust for user text where casing isn’t predictable.

Future scope:

- Enhance the regex to allow hyphenated words (e.g., J-Framework).
- Store results in a Set to avoid duplicates if needed.
- Integrate this with text analytics pipelines to extract keywords automatically.

Scenario 13: Reversing the order of words in a sentence

Let’s say in a messaging app, a user types "Java is great", and we want to reverse the word order so that it becomes "great is Java". Notice that the characters inside each word stay the same; only the word sequence changes.

Answer:

The approach is to split the sentence into words using split(" "). This gives us an array of words. Then, iterate over the array in reverse order and append each word into a StringBuilder. Finally, trim any trailing space from the result.

Code:

```
public class WordsReverser {
    public static void main(String[] args) {
        String sentence = "Java is great";
        String[] wordArray = sentence.split(" ");

        StringBuilder reversed = new StringBuilder();
        for (int i = wordArray.length - 1; i >= 0; i--) {
            reversed.append(wordArray[i]).append(" ");
        }

        System.out.println(reversed.toString().trim());
        // Output: great is Java
    }
}
```

Reasoning to interviewer:

- Splitting on spaces gives a clean list of words.
- Using a backward loop ensures we reverse the order efficiently.
- StringBuilder avoids creating multiple intermediate String objects.
- .trim() removes the extra space at the end, giving a clean result.

Future scope:

- Handle punctuation by splitting with regex like split("\\s+") to capture multiple spaces.
- Add localization support for sentences in other languages where word separation might differ.

- Wrap this into a reusable utility method for text processing features.

Scenario 14: Detecting if a string contains numeric digits

Consider a signup form where a username should not contain numbers. We need a quick way to detect if a string has digits.

Answer:

We can use regex with the matches() method. The pattern .*\d.* checks if there's at least one digit anywhere in the string.

- \d matches a digit.
- .* allows any characters before or after the digit.

Code:

```
public class ContainsDigits {
    public static void main(String[] args) {
        String text = "Java 123 programming!";
        boolean containsDigits = text.matches(".*\\d.*");

        if (containsDigits) {
            System.out.println("The string contains digits.");
        } else {
            System.out.println("The string does not contain digits.");
        }
    }
}
```

Reasoning to interviewer:

- matches() applies the regex to the entire string, and .*\d.* ensures at least one digit is present anywhere.
- This is concise and avoids writing manual loops to check each character.
- It's also easy to extend—for example, replacing \d with [0-9]{2,} could check for two or more digits.

Future scope:

- Use Pattern.compile(".*\\d.*") when checking multiple strings to avoid recompiling regex each time.
- Extend regex to validate specific numeric rules, like "string must start with a digit".

- For performance with very large text, consider using chars().anyMatch(Character::isDigit).

Scenario 15: Extracting all email addresses from text files

Imagine you're building a log parser for a customer support system, and you need to scan through large text files to extract all email addresses. These could appear in different formats like support@example.com or user.name@sub.domain.org.

Answer:

The general regex for matching email addresses is:

\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b

- \b ensures word boundaries.
- [A-Za-z0-9._%+-]+ matches the username part.
- @[A-Za-z0-9.-]+ covers domains and subdomains.
- \.[A-Za-z]{2,} checks for a top-level domain with at least two letters.

Code:

```
import java.util.regex.*;
import java.io.*;

public class ExtractEmails {
    public static void main(String[] args) throws IOException {
        String text = "Contact us at support@example.com or sales@domain.org.";
        String regex = "\\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}\\b";

        Pattern pattern = Pattern.compile(regex);
        Matcher matcher = pattern.matcher(text);

        while (matcher.find()) {
            System.out.println("Found email: " + matcher.group());
        }
    }
}
```

Reasoning to interviewer:

- The regex is flexible enough to catch common email patterns, including subdomains.
- Using matcher.find() in a loop ensures we don't stop at the first match, but extract all occurrences.
- This approach works well for in-memory text, and with a BufferedReader we can scale it to very large files.

Future scope:

- Integrate validation using javax.mail.internet.InternetAddress for RFC compliance.
- Store extracted emails into a Set to avoid duplicates.
- Use streaming I/O (BufferedReader.lines()) for large-scale log or document processing.

Scenario 16: Extracting paragraphs containing a keyword (case-insensitive)

Suppose you're working on a research document processor where each paragraph is separated by newlines, and you need to pull out only those paragraphs that mention "Java", regardless of casing ("Java", "JAVA", "java").

Answer:

We can compile a regex with the (?i) flag for case-insensitivity. Then, process the file line by line using a BufferedReader, check each paragraph against the regex, and collect matches. Pattern.quote(keyword) is used to safely escape any special regex characters in the keyword.

Code:

```
import java.io.*;
import java.util.regex.*;

public class ParagraphExtractor {
    public static void main(String[] args) throws IOException {
        String filePath = "paragraph1.txt"; // File path of the document
        String keyword = "Java";
        String regex = "(?i).*" + Pattern.quote(keyword) + ".*";
        Pattern pattern = Pattern.compile(regex);

        try (BufferedReader reader = new BufferedReader(new FileReader(filePath)))
{
            String paragraph;
            while ((paragraph = reader.readLine()) != null) {
                Matcher matcher = pattern.matcher(paragraph);
                if (matcher.find()) {
                    System.out.println("Matching paragraph: " + paragraph);
                }
            }
        }
    }
}
```

Reasoning to interviewer:

- The (?i) flag removes the need to lowercase the text manually.
- .* ensures we can match the keyword anywhere within the paragraph.
- Pattern.quote() prevents issues if the keyword itself contains regex meta characters (like . or +).
- Using BufferedReader makes the code memory-efficient for large documents.

Future scope:

- Extend to handle multi-line paragraphs separated by blank lines, not just single lines.
- Collect results into a list for further processing (like highlighting or indexing).
- Support multiple keywords by building a combined regex with alternation ((word1|word2|word3)).

Scenario 17: Extracting URLs from text

Suppose you're developing a web scraper or a content moderation system where users might paste text containing links like http://example.com, https://www.example.org, or ftp://ftp.example.com. The task is to reliably extract all such URLs.

Answer:

A regex can capture common URL formats. For HTTP/HTTPS, the pattern could be:

https?://[a-zA-Z0-9.-]+(?:/[a-zA-Z0-9&%_./-]*)?

- https? matches either http or https.
- :// ensures the scheme delimiter.
- [a-zA-Z0-9.-]+ matches the domain name.
- The non-capturing group (?:/[a-zA-Z0-9&%_./-]*)? optionally matches paths after the domain.

Code:

```
import java.util.regex.*;

public class URLExtractor {
    public static void main(String[] args) {
        String input = "Visit http://example.com or https://www.example.org for info.
ftp://ftp.example.com is also accessible.";
        String regex = "https?://[a-zA-Z0-9.-]+(?:/[a-zA-Z0-9&%_./-]*)?";

        Pattern pattern = Pattern.compile(regex);
        Matcher matcher = pattern.matcher(input);

        while (matcher.find()) {
            System.out.println("URL found: " + matcher.group());
        }
    }
}
```

Reasoning to interviewer:

- The regex is compact yet flexible enough to detect both HTTP and HTTPS links with or without paths.
- matcher.find() allows scanning through the entire text, ensuring all URLs are extracted.

- This approach is useful in scenarios like link validation, scraping, or analytics pipelines.

Future scope:

- Extend regex to handle ftp:// and other schemes by generalizing the pattern to (https?|ftp)://....
- Add support for query parameters (?key=value) and fragments (#section).
- For production-grade systems, rely on java.net.URL or third-party libraries (like Apache Commons Validator) for stricter parsing.

Scenario 18: Validating product SKU format

Think of an e-commerce inventory system where each product has a SKU in the format AB-1234-5678. We need to strictly validate input to prevent incorrectly formatted SKUs from entering the system.

Answer:

The regex for this format is:

^[A-Z]{2}-\d{4}-\d{4}$

- ^[A-Z]{2} → exactly two uppercase letters at the start.
- - → hyphen separator.
- \d{4} → exactly four digits.
- Another - and \d{4} → another four digits at the end.
- $ ensures the match ends there.

Code:

```
import java.util.regex.*;

public class SKUValidator {
    public static void main(String[] args) {
        String sku = "AB-1234-5678";
        String patternStr = "^[A-Z]{2}-\\d{4}-\\d{4}$";

        Pattern pattern = Pattern.compile(patternStr);
        Matcher matcher = pattern.matcher(sku);

        if (matcher.matches()) {
            System.out.println("SKU is valid: " + sku);
        } else {
            System.out.println("SKU format is invalid.");
        }
    }
}
```

Reasoning to interviewer:

- This regex enforces the SKU format rigidly—no extra characters, lowercase letters, or misplaced digits are allowed.
- Using matcher.matches() ensures the *entire* string matches the pattern, not just a part of it.

- It’s efficient and clear for input validation before saving to the database.

Future scope:

- Extend the pattern to handle variable-length SKU segments (if business rules change).
- Localize SKU rules for different regions (e.g., allowing lowercase or additional prefixes).
- Add this logic as part of a validator utility in the product catalog service, ensuring consistent checks across the application.

Scenario 19: Extracting words or phrases inside square brackets

Let's say you're processing a customer support document where certain tags are highlighted inside square brackets like [important], [urgent], or [follow-up]. The task is to extract just the content inside those brackets for further processing, like categorization or priority tagging.

Answer:

A regex like \[(.*?)\] works well here.

- \[matches the opening bracket literally.
- (.*?) is a non-greedy group that captures all characters inside the brackets.
- \] matches the closing bracket.

  Using matcher.group(1) gives us only the inside text without the brackets.

Code:

```
import java.util.regex.*;

public class BracketExtractor {
    public static void main(String[] args) {
        String text = "This is [important] and this is [urgent].";
        String regex = "\\[(.*?)\\]";

        Pattern pattern = Pattern.compile(regex);
        Matcher matcher = pattern.matcher(text);

        while (matcher.find()) {
            System.out.println("Extracted text: " + matcher.group(1));
        }
    }
}
```

Reasoning to interviewer:

- The .*? part ensures non-greedy matching—this avoids capturing across multiple bracketed sections in one go.
- matcher.find() allows scanning through the entire text and retrieving each match.
- matcher.group(1) gives only the inner content, which is usually what's useful for processing.

Future scope:

- Enhance regex to handle nested brackets if the business case allows [tag [inner]].
- Store extracted tags into a Set to ensure uniqueness before indexing.
- Build a tagging utility class that can categorize documents based on extracted keywords.

Scenario 20: Case-insensitive search for multiple keywords in any order

Imagine you're building a document search feature. A user might type "java python", and your system should highlight all occurrences of "java" or "python", regardless of case and no matter where they appear in the text.

Answer:

We can dynamically build a regex by joining the keywords with the | operator (OR). Prepending (?i) makes the pattern case-insensitive. Then we use matcher.find() to iterate through matches.

Code:

```
import java.util.regex.*;

public class KeywordSearch {
    public static void main(String[] args) {
        String text = "Java is great, and JavaScript is awesome!";
        String[] keywords = {"java", "javascript", "python"};

        StringBuilder regex = new StringBuilder("(?i)"); // case-insensitive flag
        for (String keyword : keywords) {
            regex.append(keyword).append("|");
        }
        regex.deleteCharAt(regex.length() - 1); // remove last '|'

        Pattern pattern = Pattern.compile(regex.toString());
        Matcher matcher = pattern.matcher(text);

        while (matcher.find()) {
            System.out.println("Keyword found: " + matcher.group());
        }
    }
}
```

Reasoning to interviewer:

- Using (?i) saves us from manually converting both text and keywords to lowercase.
- Building the regex dynamically means we can handle any number of keywords.
- matcher.find() scans through the text, ensuring all occurrences of all keywords are captured.

Future scope:

- Escape special regex characters in keywords with Pattern.quote(keyword).
- Highlight or replace keywords in the original text using matcher.appendReplacement().
- For huge datasets, integrate with indexing/search libraries like Lucene or ElasticSearch.

Scenario 21: Extracting hashtags from a tweet

Suppose you're working on a social media analytics tool. You need to extract hashtags from tweets, e.g., "Loving #Java and exploring #JavaScript! #100DaysOfCode".

Answer:

We can use the regex #\w+.

- # matches the literal hash symbol.
- \w+ matches one or more word characters (letters, numbers, or underscores).

Code:

```
import java.util.regex.*;

public class HashtagExtractor {
    public static void main(String[] args) {
        String tweet = "Loving #Java and exploring #JavaScript! #100DaysOfCode";
        String regex = "#\\w+";

        Pattern pattern = Pattern.compile(regex);
        Matcher matcher = pattern.matcher(tweet);

        while (matcher.find()) {
            System.out.println("Hashtag: " + matcher.group());
        }
    }
}
```

Reasoning to interviewer:

- The regex is simple but powerful—it captures all hashtags as long as they follow the #word format.
- Using matcher.find() ensures we collect every hashtag, not just the first one.
- Works for hashtags like #Java, #100DaysOfCode, or #Dev_Tools.

Future scope:

- Support hashtags in other languages by enhancing regex with Unicode categories (\p{L} for letters).
- Store hashtags in a Set to avoid duplicates before processing.
- Extend extraction logic to also capture trending hashtag frequencies.

Scenario 22: Extracting ERROR events from log files

Imagine you're working on a server monitoring system. The log file contains entries like INFO, DEBUG, and ERROR. The requirement is to filter out and display only the error-related events so that developers can focus on troubleshooting.

Answer:

We can use the regex ^ERROR.* to capture lines starting with "ERROR".

- ^ anchors the match to the beginning of the line.
- ERROR looks for the literal string.
- .* allows any characters after "ERROR".

  Reading the file line by line with a BufferedReader ensures scalability for large logs.

Code:

```
import java.io.*;
import java.util.regex.*;

public class ErrorExtractor {
    public static void main(String[] args) throws IOException {
        String logFilePath = "logfile.txt";
        String regex = "^ERROR.*";

        Pattern pattern = Pattern.compile(regex);

        try (BufferedReader br = new BufferedReader(new FileReader(logFilePath)))
{
            String line;
            while ((line = br.readLine()) != null) {
                Matcher matcher = pattern.matcher(line);
                if (matcher.find()) {
                    System.out.println("ERROR event: " + line);
                }
            }
        }
    }
}
```

Reasoning to interviewer:

- Anchoring with ^ ensures we only pick lines that *start* with "ERROR", not those that just mention it in the middle.

- Processing line by line avoids memory issues when handling huge log files.
- This pattern is flexible—you can tweak it to catch WARN or DEBUG events in the same way.

Future scope:

- Enhance regex to capture timestamps or error codes in separate groups.
- Feed extracted errors into a monitoring dashboard for real-time alerts.
- Write results to a separate file for easier debugging workflows.

Scenario 23: Normalizing phone numbers into a standard format

Consider a customer service database where phone numbers arrive in mixed formats like "123-456-7890", "(123) 456-7890", or "123.456.7890". The goal is to unify them into a clean, consistent format (XXX) XXX-XXXX.

Answer:
We can design a regex that handles optional parentheses and different separators (-, ., or space). After extraction, remove all non-digit characters, then reformat the digits using String.format().

Code:

```
import java.util.regex.*;

public class PhoneNormalizer {
    public static void main(String[] args) {
        String sampleText = "Reach us at (123) 456-7890, 123.456.7890, or 123-456-
7890.";
        String phonePattern = "(?:\\(?\\d{3}\\)?[-\\.\\s]?\\d{3}[-\\.\\s]?\\d{4})";

        Pattern compiledPattern = Pattern.compile(phonePattern);
        Matcher matcher = compiledPattern.matcher(sampleText);

        while (matcher.find()) {
            String rawNumber = matcher.group();

            // Remove non-digit characters
            rawNumber = rawNumber.replaceAll("[^\\d]", "");

            // Reformat into (XXX) XXX-XXXX
            String formatted = String.format("(%s) %s-%s",
                    rawNumber.substring(0, 3),
                    rawNumber.substring(3, 6),
                    rawNumber.substring(6));

            System.out.println("Standardized phone: " + formatted);
        }
    }
}
```

Reasoning to interviewer:

- The regex \(?\d{3}\)?[-.\s]?\d{3}[-.\s]?\d{4} flexibly handles variations in input format.

- Stripping all non-digits ensures we work with a clean 10-digit string.
- String.format() makes the final representation consistent and readable.

Future scope:

- Add support for country codes like +1 or +91.
- Store results in normalized form for database consistency.
- Build a reusable utility class for validating and formatting phone numbers across the system.

Scenario 24: Extracting and categorizing log messages by severity

Suppose you're analyzing logs coming from different services. Each entry follows a simple structure like:

- ERROR: Disk space is full
- WARN: CPU usage is high
- INFO: System running smoothly

You want to extract both the severity level and the message for further processing or alerting.

Answer:

We can use the regex:

(ERROR|WARN|INFO):\s+(.*)

- (ERROR|WARN|INFO) captures the severity keyword.
- :\s+ matches the colon and following spaces.
- (.*) captures the rest of the message text.

Code:

```
import java.util.regex.*;

public class LogMessageParser {
    public static void main(String[] args) {
        String logs = "ERROR: Disk space is full\n"
                    + "WARN: CPU usage is high\n"
                    + "INFO: System running smoothly";

        String regex = "(ERROR|WARN|INFO):\\s+(.*)";
        Pattern pattern = Pattern.compile(regex);
        Matcher matcher = pattern.matcher(logs);

        while (matcher.find()) {
            String level = matcher.group(1);  // severity type
            String msg = matcher.group(2);    // message content
            System.out.println("Level: " + level + ", Message: " + msg);
        }
    }
}
```

Reasoning to interviewer:

- Group 1 holds the severity, which is useful for categorization.
- Group 2 holds the actual message text.
- This approach makes it easy to route ERROR logs to alerts, WARN logs to warnings dashboard, and keep INFO for monitoring.

Future scope:

- Extend regex to also capture timestamps if logs include them.
- Store parsed logs into a database for analytics.
- Add filtering logic to process only a specific severity level.

Scenario 25: Escaping special regex characters in keywords

Imagine you're building a keyword search for user-generated content. A user might search for "price+discount", but since + is a regex operator, it would cause unexpected matches unless escaped.

Answer:

In Java, regex metacharacters like +, *, ?, and . need escaping. Instead of escaping manually (which is error-prone), Java provides Pattern.quote(). It wraps the keyword so it's treated as a literal string in regex.

Code:

```
import java.util.regex.*;

public class SpecialCharEscaper {
    public static void main(String[] args) {
        String keyword = "price+discount";
        String text = "The price+discount is valid today.";

        String escapedKeyword = Pattern.quote(keyword);
        Pattern pattern = Pattern.compile(escapedKeyword);
        Matcher matcher = pattern.matcher(text);

        if (matcher.find()) {
            System.out.println("Keyword found: " + matcher.group());
        }
    }
}
```

Reasoning to interviewer:

- Pattern.quote() automatically escapes all regex metacharacters, so "price+discount" is treated literally.
- This avoids bugs where keywords unintentionally act like regex operators.
- It makes the code safe for dynamic user input where you don't control what characters appear.

Future scope:

- Combine with Pattern.CASE_INSENSITIVE if search needs to ignore case.
- Use this in keyword highlighting utilities to display matches in UI.

- For multiple keywords, build a combined regex using Pattern.quote() for each term.

Scenario 26: Validating multiple date formats with regex

In real systems, users often enter dates in different formats depending on region or habit. For example:

- "2021-12-25" (ISO style)
- "12/25/2021" (US style)
- "25-12-2021" (European style)

The task is to validate whether the input matches any of these expected structures.

Answer:

We can use separate regexes for each format, then combine them using the alternation (|) operator:

- \d{4}-\d{2}-\d{2} → yyyy-MM-dd
- \d{2}/\d{2}/\d{4} → MM/dd/yyyy
- \d{2}-\d{2}-\d{4} → dd-MM-yyyy

Code:

```
import java.util.regex.*;

public class MultiFormatDateValidator {
    public static void main(String[] args) {
        String inputDate = "25-12-2021";
        String datePattern = "(\\d{4}-\\d{2}-\\d{2}|\\d{2}/\\d{2}/\\d{4}|\\d{2}-\\d{2}-\\d{4})";

        Pattern pattern = Pattern.compile(datePattern);
        Matcher matcher = pattern.matcher(inputDate);

        if (matcher.matches()) {
            System.out.println("Date format is valid: " + inputDate);
        } else {
            System.out.println("Date format is invalid.");
        }
    }
}
```

Reasoning to interviewer:

- Alternation allows one regex to cover multiple formats.

- matcher.matches() ensures the *entire* input string is validated, not just a substring.
- This approach is good for structure validation, but not for semantic correctness (e.g., it won't catch February 30).

Future scope:

- After structural validation, pass the string to LocalDate.parse with multiple formatters to validate actual calendar correctness.
- Extend regex to support other formats (like dd/MM/yyyy).
- Add user-specific locale preferences to automatically detect expected formats.

Scenario 27: Counting pattern occurrences in a large file

Suppose you're analyzing a 2GB log file to count how many times the word "error" appears. Loading the entire file into memory would be inefficient, so you need a streaming approach.

Answer:

The efficient way is to process the file line by line using a BufferedReader. Compile the regex pattern once, then for each line, run matcher.find() and increment a counter whenever a match is found.

Code:

```
import java.io.*;
import java.util.regex.*;

public class PatternOccurrenceCounter {
    public static void main(String[] args) throws IOException {
        String filePath = "large_text.txt";
        String searchPattern = "error";

        Pattern pattern = Pattern.compile(searchPattern,
Pattern.CASE_INSENSITIVE);
        int occurrences = 0;

        try (BufferedReader reader = new BufferedReader(new FileReader(filePath)))
{
            String currentLine;
            while ((currentLine = reader.readLine()) != null) {
                Matcher matcher = pattern.matcher(currentLine);
                while (matcher.find()) {
                    occurrences++;
                }
            }
        }

        System.out.println("Total occurrences of pattern: " + occurrences);
    }
}
```

Reasoning to interviewer:

- BufferedReader prevents memory overload when handling huge files.
- Pre-compiling the regex improves performance instead of recompiling on each line.

- Using matcher.find() ensures all matches in each line are counted.

Future scope:

- Extend to count multiple patterns simultaneously using a Map<String, Integer>.
- Use parallel streams or multi-threaded file processing for even faster counts on massive files.
- If the pattern is very simple (like a fixed word), using String.indexOf inside a loop may be faster than regex.

Scenario 28: Extracting email addresses from text

Imagine you're building an email indexing feature for a customer support system. Incoming text may contain email addresses embedded inside paragraphs, such as "Please contact us at support@example.com or admin@domain.org". The requirement is to detect and extract all valid emails.

Answer:

We can use a regex pattern to capture the structure of standard email addresses:

[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}

- [a-zA-Z0-9._%+-]+ → matches the username part before @.
- @[a-zA-Z0-9.-]+ → matches the domain name.
- \.[a-zA-Z]{2,} → ensures a valid top-level domain with at least two letters.

Code:

```
import java.util.regex.*;

public class EmailFinder {
    public static void main(String[] args) {
        String text = "Reach us at support@example.com or admin@domain.org.";
        String regex = "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}";

        Pattern pattern = Pattern.compile(regex);
        Matcher matcher = pattern.matcher(text);

        while (matcher.find()) {
            System.out.println("Email found: " + matcher.group());
        }
    }
}
```

Reasoning to interviewer:

- The regex captures common email formats without overcomplication.
- Using matcher.find() ensures that *all* email addresses in the text are extracted, not just the first one.
- This approach is flexible for embedded content, such as parsing emails from chat logs or documents.

Future scope:

- Use stricter regexes or javax.mail.internet.InternetAddress for RFC-compliant validation.
- Store emails in a Set to avoid duplicates before processing.
- Combine extraction with masking (e.g., partially hiding addresses before displaying).

Scenario 29: Validating product SKUs of format XXX-YYYY-ZZZZ

Suppose you're managing inventory in an e-commerce application. Each product SKU must strictly follow a format like "ABC-1234-5678", where:

- The first three characters are alphanumeric (XXX).
- Followed by a hyphen.
- Then four alphanumeric characters (YYYY).
- Another hyphen.
- Finally, four digits (ZZZZ).

Answer:

We can enforce this using the regex:

^[A-Za-z0-9]{3}-[A-Za-z0-9]{4}-\d{4}$

- ^[A-Za-z0-9]{3} → three alphanumeric characters at the start.
- -[A-Za-z0-9]{4} → a hyphen followed by four alphanumeric characters.
- -\d{4}$ → a hyphen followed by exactly four digits at the end.

Code:

```
import java.util.regex.*;

public class SKUChecker {
    public static void main(String[] args) {
        String sku = "ABC-1234-5678";
        String regex = "^[A-Za-z0-9]{3}-[A-Za-z0-9]{4}-\\d{4}$";

        Pattern pattern = Pattern.compile(regex);
        Matcher matcher = pattern.matcher(sku);

        if (matcher.matches()) {
            System.out.println("SKU is valid: " + sku);
        } else {
            System.out.println("SKU is invalid.");
        }
    }
}
```

Reasoning to interviewer:

- The regex enforces the SKU format strictly, ensuring no extra characters creep in.
- matcher.matches() checks the *entire* string against the format, not just part of it.
- This prevents invalid SKUs like "AB-12345-5678" or "ABC-123-XYZ".

Future scope:

- Extend the format to allow region codes or vendor prefixes.
- Use regex groups to extract different SKU segments for categorization.
- Add batch validation for multiple SKUs read from a file or API feed.

Scenario 30: Extracting and validating email addresses from large CSV files

Suppose you're working with customer data stored in CSV files containing millions of rows. Each row has user details like name, email, and phone number. The task is to extract the email column, check if each entry is valid, and keep only the valid ones for downstream processing like newsletters or account creation.

Answer:

The efficient way is to combine regex-based validation with streaming file reads:

1. Use BufferedReader to read the file line by line, avoiding memory overload.
2. Assume the email is in a known column (e.g., the second column). Split the line accordingly.
3. Use a compiled regex to validate the email format.

Code:

```
import java.io.*;
import java.util.regex.*;

public class EmailExtractor {
    public static void main(String[] args) throws IOException {
        String filePath = "user_data.csv";
        String regex = "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$";

        Pattern pattern = Pattern.compile(regex);

        try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {
            String line;
            while ((line = br.readLine()) != null) {
                // Assuming email is in the second column
                String[] columns = line.split(",");
                if (columns.length > 1) {
                    String email = columns[1].trim();
                    Matcher matcher = pattern.matcher(email);
                    if (matcher.matches()) {
                        System.out.println("Valid email: " + email);
                    }
                }
            }
        }
    }
}
```

Reasoning to interviewer:

- BufferedReader ensures scalability for very large CSV files.
- Regex validation is fast enough for structural checks.
- Precompiling the regex improves performance compared to recompiling for every row.
- This approach isolates only valid emails, preventing garbage data from entering the system.

Future scope:

- Use libraries like OpenCSV for robust CSV parsing (handles quotes, escapes).
- Add semantic validation like checking DNS MX records for domains.
- Write valid emails to a new file or database instead of just printing them.

Scenario 31: Normalizing product names for consistency

In an e-commerce platform, product names often arrive messy, e.g., " large plastic chair " or "WOODEN taBLE". For a clean UI and better search indexing, product names should have consistent spacing and capitalization—“Large Plastic Chair”, “Wooden Table”.

Answer:

The steps are:

1. Trim leading and trailing spaces.
2. Replace multiple spaces between words with a single space.
3. Split into words and capitalize each word (first letter uppercase, rest lowercase).

Code:

```
public class NormalizeProductName {
    public static void main(String[] args) {
        String productName = " large plastic chair ";

        // Step 1 & 2: Trim and normalize spaces
        productName = productName.trim().replaceAll("\\s+", " ");

        // Step 3: Capitalize each word
        String[] words = productName.split(" ");
        StringBuilder normalizedName = new StringBuilder();

        for (String word : words) {
            normalizedName.append(word.substring(0, 1).toUpperCase())
                          .append(word.substring(1).toLowerCase())
                          .append(" ");
        }

        System.out.println(normalizedName.toString().trim());
        // Output: Large Plastic Chair
    }
}
```

Reasoning to interviewer:

- trim() clears unnecessary spaces at boundaries.
- replaceAll("\\s+", " ") compresses multiple spaces to one, making splitting reliable.

- Processing each word ensures consistent title case formatting, regardless of original casing.

Future scope:

- Handle edge cases like acronyms (USB, HDMI) where capitalization rules differ.
- Integrate into a product catalog pipeline so all names are normalized before display.
- Support localization rules for different languages when capitalizing words.

Scenario 32: Extracting timestamps from log entries

Consider a server log file where each entry starts with a timestamp like "2025-09-13 14:45:32 INFO Application started". The requirement is to pull out all timestamps in the format yyyy-MM-dd HH:mm:ss.

Answer:

We can use the regex:

\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}

- \d{4}-\d{2}-\d{2} → matches the date part.
- A space separates date and time.
- \d{2}:\d{2}:\d{2} → matches the time part.

Code:

```
import java.util.regex.*;
import java.io.*;

public class LogTimestampExtractor {
    public static void main(String[] args) throws IOException {
        String logFile = "logs.txt";
        String regex = "\\d{4}-\\d{2}-\\d{2} \\d{2}:\\d{2}:\\d{2}";

        Pattern pattern = Pattern.compile(regex);

        try (BufferedReader br = new BufferedReader(new FileReader(logFile))) {
            String line;
            while ((line = br.readLine()) != null) {
                Matcher matcher = pattern.matcher(line);
                if (matcher.find()) {
                    System.out.println("Timestamp: " + matcher.group());
                }
            }
        }
    }
}
```

Reasoning to interviewer:

- This regex tightly matches the standard yyyy-MM-dd HH:mm:ss structure.
- Using BufferedReader ensures the solution works well even for gigabyte-scale log files.

- matcher.find() is used instead of matches() since the timestamp might be just part of a larger log entry.

Future scope:

- Extend regex to also capture milliseconds if logs include yyyy-MM-dd HH:mm:ss.SSS.
- Convert extracted strings into LocalDateTime objects for time-based filtering.
- Handle multiple timestamp formats if logs come from heterogeneous systems.

Scenario 33: Extracting text inside double quotes

Imagine you're parsing a document where phrases of interest are enclosed in quotes, such as:
This is a "sample" text with "quoted" words.
You need to capture only the text inside the quotation marks.

Answer:

The regex pattern for this is:

"(.*?)"

- " → matches the literal double-quote.
- (.*?) → captures everything inside, using non-greedy matching so it stops at the first closing quote.
- Another " closes the pattern.

Code:

```
import java.util.regex.*;

public class ExtractQuotedText {
    public static void main(String[] args) {
        String text = "This is a \"sample\" text with \"quoted\" words.";
        String regex = "\"(.*?)\"";

        Pattern pattern = Pattern.compile(regex);
        Matcher matcher = pattern.matcher(text);

        while (matcher.find()) {
            System.out.println("Extracted text: " + matcher.group(1));
        }
    }
}
```

Reasoning to interviewer:

- Non-greedy matching (*?) prevents the regex from grabbing everything between the first opening quote and the last closing quote.
- matcher.group(1) ensures we only capture the inner content, excluding the quotes themselves.

- This works well for extracting quoted phrases from structured or semi-structured text.

Future scope:

- Modify regex to also handle escaped quotes (\") if present inside text.
- Apply the same logic to extract content enclosed in other delimiters, like parentheses or brackets.
- Build a utility to highlight quoted text in documents for downstream NLP or indexing tasks.

Scenario 34: Validating a URL string with regex

Think of a web crawler or CMS where users submit links. You want to confirm that a submitted string is a properly formatted URL before processing it further. It should start with http:// or https://, contain a valid domain, and optionally include a path.

Answer:

We can use a regex such as:

^(https?://)?[a-zA-Z0-9.-]+(?:/[a-zA-Z0-9&%_./-]*)?$

- ^(https?://)? → ensures it either starts with http:// or https://, but makes it optional.
- [a-zA-Z0-9.-]+ → checks for a valid domain name.
- (?:/[a-zA-Z0-9&%_./-]*)? → optionally allows a path with characters like /, _, . or -.
- $ → ensures the whole string conforms.

Code:

```
import java.util.regex.*;

public class URLValidator {
    public static void main(String[] args) {
        String urlToTest = "https://www.example.com/path/to/resource";
        String urlPattern = "^(https?://)?[a-zA-Z0-9.-]+(?:/[a-zA-Z0-9&%_./-]*)?$";

        Pattern pattern = Pattern.compile(urlPattern);
        Matcher matcher = pattern.matcher(urlToTest);

        if (matcher.matches()) {
            System.out.println("URL is valid.");
        } else {
            System.out.println("URL is invalid.");
        }
    }
}
```

Reasoning to interviewer:

- This regex is lightweight and covers the majority of real-world URLs.
- matcher.matches() forces the check against the entire string, avoiding partial matches.

- It ensures users don’t accidentally submit invalid or malformed links.

Future scope:

- Extend regex to cover query parameters (?id=123) and anchors (#section).
- Use java.net.URL or URI classes for strict parsing beyond structural checks.
- Integrate into form validation or link extraction pipelines.

Scenario 35: Efficient multiple find-and-replace in large text

Imagine a content management system where an article body contains multiple terms that need replacing — e.g., replacing "Java" with "Python" and "JavaScript" with "C#" — across very large text. Running replaceAll() separately for each term can be costly.

Answer:

The straightforward way is to use a Map<String, String> to store patterns and replacements, then iterate over it and apply replacements. For small to medium workloads this is fine. For larger workloads, you can optimize further with combined regex patterns and a single pass replacement using Matcher.appendReplacement().

Code (practical approach):

```
import java.util.*;
import java.util.regex.*;

public class MultiPatternReplacer {
    public static void main(String[] args) {
        String content = "Java is powerful. JavaScript is amazing!";

        Map<String, String> replacementMap = new HashMap<>();
        replacementMap.put("Java", "Python");
        replacementMap.put("JavaScript", "C#");

        for (Map.Entry<String, String> entry : replacementMap.entrySet()) {
            content = content.replaceAll(entry.getKey(), entry.getValue());
        }

        System.out.println(content); // Output: Python is powerful. C# is amazing!
    }
}
```

Reasoning to interviewer:

- Using a Map keeps replacements organized and easier to manage.
- Iterating through the map avoids scattering multiple replaceAll() calls in the code.
- Although this approach scans the text once per pattern, it’s still efficient for most applications.

Future scope:

- For very large text and many patterns, combine keys into a single regex (e.g., (Java|JavaScript)) and replace dynamically using Matcher.appendReplacement().
- Use libraries like Apache Commons Text StringSubstitutor for more complex template-based replacements.
- Streamline replacements with preprocessing if patterns are known in advance and frequent.

Scenario 36: Checking if a string starts with a specific word (case-insensitive)

Suppose you're building a chatbot or command parser where user input like "Hello, bot!" or "hello world" should be recognized as starting with "hello", regardless of capitalization.

Answer:

A simple approach is to normalize both the input string and the target word to the same case (lowercase or uppercase), then use startsWith().

Code:

```
public class StartWordChecker {
    public static void main(String[] args) {
        String input = "Hello, welcome to Java!";
        String targetWord = "hello";

        if (input.toLowerCase().startsWith(targetWord.toLowerCase())) {
            System.out.println("The string starts with 'hello'.");
        } else {
            System.out.println("The string does not start with 'hello'.");
        }
    }
}
```

Reasoning to interviewer:

- toLowerCase() ensures case-insensitive comparison without changing the logic.
- startsWith() cleanly checks if the input begins with the target word.
- This method is efficient and avoids regex overhead for simple prefix checks.

Future scope:

- Use toLowerCase(Locale.ROOT) for consistent results across locales (important for languages like Turkish where "I".toLowerCase() behaves differently).
- If multiple prefixes need checking (e.g., "hi", "hello", "hey"), store them in a list and iterate.
- For more complex matching (like natural language commands), regex or NLP tokenization can be used.

Scenario 37: Validating international phone numbers

Suppose you're designing a customer onboarding system for a global platform. Users may enter phone numbers in different formats like:

- +1 (123) 456-7890
- 0044 20 7946 0958
- (022) 1234-5678

The requirement is to validate these diverse formats while accounting for optional country codes, separators, and parentheses.

Answer:

We can use a regex like:

(?:\+\d{1,3}|00\d{1,3})?[-\s]?\(?\d{1,4}\)?[-\s]?\d{1,4}[-\s]?\d{1,4}

- (?:\+\d{1,3}|00\d{1,3})? → optional country code, either with + or 00 prefix.
- \(?\d{1,4}\)? → optional area code wrapped in parentheses.
- [-\s]? → allows dashes or spaces as separators.
- Remaining groups of \d{1,4} allow flexibility for local number formats.

Code:

```
import java.util.regex.*;

public class InternationalPhoneValidator {
    public static void main(String[] args) {
        String phoneNumber = "+1 (123) 456-7890";
        String phonePattern = "(?:\\+\\d{1,3}|00\\d{1,3})?[-\\s]?\\(?\\d{1,4}\\)?[-
\\s]?\\d{1,4}[-\\s]?\\d{1,4}";

        Pattern pattern = Pattern.compile(phonePattern);
        Matcher matcher = pattern.matcher(phoneNumber);

        if (matcher.matches()) {
            System.out.println("Phone number is valid.");
        } else {
            System.out.println("Phone number is invalid.");
        }
    }
}
```

Reasoning to interviewer:

- This regex covers common global formats without being overly strict.
- matcher.matches() ensures the entire string matches, preventing partial matches like "Call me at 123".
- It strikes a balance between flexibility (for international variations) and structure (ensuring digits + separators are in the right places).

Future scope:

- Integrate libraries like Google's libphonenumber for comprehensive validation, since regex alone can't verify number validity for specific countries.
- Normalize valid numbers into E.164 format (+11234567890) for consistent storage.
- Extend to handle extensions (e.g., +1-123-456-7890 x123).

Scenario 38: Extracting words of a specific length from text

Imagine you're building a text-analysis tool that needs to pull out all words of exactly 5 characters from a paragraph for linguistic processing. For example, from "apple and table", the extracted words should be "apple" and "table".

Answer:

The regex for this is:

\b\w{5}\b

- \b → word boundary ensures we only match complete words.
- \w{5} → matches exactly five word characters (letters, digits, or underscores).

Code:

```
import java.util.regex.*;

public class WordsByLengthExtractor {
    public static void main(String[] args) {
        String input = "This is a sample text with some words like apple and table.";
        String pattern = "\\b\\w{5}\\b"; // words with exactly 5 characters

        Pattern compiledPattern = Pattern.compile(pattern);
        Matcher matcher = compiledPattern.matcher(input);

        while (matcher.find()) {
            System.out.println("Found word: " + matcher.group());
        }
    }
}
```

Reasoning to interviewer:

- Using word boundaries prevents partial matches inside longer words.
- This approach works even if punctuation surrounds words (like "apple,") because boundaries detect transitions between word and non-word characters.
- Regex provides a concise solution compared to manually splitting and checking lengths.

Future scope:

- Adjust pattern to capture variable ranges like \w{3,7} for words between 3–7 characters.
- Extend to match only alphabetic words ([A-Za-z]{5}) if digits/underscores are not desired.
- Integrate with a tokenizer pipeline to extract words for NLP tasks.

Scenario 39: Checking if a filename ends with .txt

Suppose you're writing a file-upload feature where only text files are allowed. You need to verify if the uploaded filename ends with .txt.

Answer:

We can use the endsWith() method of the String class to check the suffix.

Code:

```
public class FileExtensionChecker {
    public static void main(String[] args) {
        String filename = "document.txt";

        if (filename.toLowerCase().endsWith(".txt")) {
            System.out.println("Valid text file: " + filename);
        } else {
            System.out.println("Invalid file type.");
        }
    }
}
```

Reasoning to interviewer:

- endsWith() is a simple and efficient way to check suffixes.
- Using toLowerCase() ensures case-insensitivity (so "Document.TXT" still matches).
- This avoids overcomplicating the logic with regex for a straightforward check.

Future scope:

- Extend validation to multiple file types (like .csv, .pdf).
- Combine with security checks to avoid malicious file uploads (e.g., file.txt.exe).
- Use Files.probeContentType() for MIME-type validation in production.

Scenario 40: Splitting CSV text into fields

Suppose you're processing a line from a CSV file like:
"John,Doe,30,New York"
You need to break it into separate fields.

Answer:

Use the split() method with a comma as the delimiter.

Code:

```
public class CSVSplitter {
    public static void main(String[] args) {
        String csvLine = "John,Doe,30,New York";
        String[] fields = csvLine.split(",");

        for (String field : fields) {
            System.out.println(field);
        }
    }
}
```

Reasoning to interviewer:

- split(",") cleanly separates values based on the delimiter.
- It returns an array, making it easy to index fields like fields[0] for first name.
- This is the simplest approach for well-formed CSVs without quoted commas.

Future scope:

- Handle edge cases where values contain commas inside quotes (e.g., "New York, USA").
- Use libraries like OpenCSV for robust CSV parsing.
- Trim whitespace around fields to clean up data.

Scenario 41: Safely handling empty or null strings (isEmpty() vs isBlank())

Imagine a signup form where a user leaves a field blank by entering " " (spaces). You need to distinguish between null, empty (""), and blank (spaces only).

Answer:

- isEmpty() checks if the string length is 0.
- isBlank() (Java 11+) also checks if the string contains only whitespace.
- Always check for null before calling these methods.

Code:

```
public class EmptyBlankChecker {
    public static void main(String[] args) {
        String str1 = "";
        String str2 = "   ";
        String str3 = null;

        System.out.println("str1 isEmpty: " + (str1.isEmpty()));  // true
        System.out.println("str2 isEmpty: " + (str2.isEmpty()));  // false
        System.out.println("str2 isBlank: " + (str2.isBlank()));  // true

        if (str3 == null) {
            System.out.println("str3 is null");
        }
    }
}
```

Reasoning to interviewer:

- isEmpty() only checks length, so " " isn't empty.
- isBlank() is better for validation when whitespace shouldn't count.
- Always guard against NullPointerException before using these methods.

Future scope:

- Create utility methods like StringUtils.isBlank() from Apache Commons, which handle null gracefully.
- Apply these checks in form validation layers consistently.

Scenario 42: Finding substrings (substring(), indexOf(), lastIndexOf())

Suppose you're parsing a URL like:
"https://example.com/page.html"
You need to extract just the domain name (example.com).

Answer:

Use indexOf() to find positions, then substring() to extract.

Code:

```
public class SubstringFinder {
    public static void main(String[] args) {
        String url = "https://example.com/page.html";

        int start = url.indexOf("://") + 3;   // after "https://"
        int end = url.indexOf("/", start);    // first "/" after domain

        String domain = url.substring(start, end);
        System.out.println("Domain: " + domain);  // Output: example.com
    }
}
```

Reasoning to interviewer:

- indexOf("://") locates the protocol separator.
- Adding 3 skips past it, so substring starts at the domain.
- indexOf("/", start) finds where the path begins.
- substring(start, end) extracts just the part we need.

Future scope:

- Use lastIndexOf() for cases where you want the last occurrence (e.g., file extension in a path).
- Use java.net.URL class for safer parsing of URLs.
- Build utility methods to encapsulate substring extraction logic.

Scenario 43: Why "Hello" == "Hello" might return true but new String("Hello") == new String("Hello") doesn't

Suppose you're debugging code and notice this:

```
System.out.println("Hello" == "Hello");                 // true
System.out.println(new String("Hello") == new String("Hello"));  // false
```

At first glance, it looks inconsistent — but it's actually because of how Java handles **string pooling** and object creation.

Answer:

- String literals (like "Hello") are stored in the **String pool**, so repeated literals point to the same memory reference.
- "Hello" == "Hello" returns true because both references point to the same pooled object.
- new String("Hello") forces creation of a **new object** on the heap every time, so two different objects are created → == (reference equality) returns false.
- To check content equality, always use .equals().

Code:

```
public class StringComparison {
    public static void main(String[] args) {
        String s1 = "Hello";
        String s2 = "Hello";
        String s3 = new String("Hello");
        String s4 = new String("Hello");

        System.out.println(s1 == s2);   // true (same pool reference)
        System.out.println(s3 == s4);   // false (different heap objects)
        System.out.println(s3.equals(s4)); // true (content comparison)
    }
}
```

Reasoning to interviewer:

- == compares object references, not content.
- String pool ensures memory efficiency for literals.
- new String() bypasses the pool, always allocating fresh objects.

Future scope:

- Use .intern() if you want a new String() to point back to the pool.
- Always prefer .equals() for content comparison in production.

Scenario 44: Sorting strings alphabetically using .compareTo()

Imagine you have a list of product names: ["Banana", "Apple", "Mango"]. You want to sort them alphabetically.

Answer:

The compareTo() method compares strings lexicographically based on Unicode values. It returns:

- 0 if strings are equal,
- A positive value if the calling string is greater,
- A negative value if the calling string is smaller.

Code:

```
import java.util.*;

public class StringSortExample {
    public static void main(String[] args) {
        List<String> products = Arrays.asList("Banana", "Apple", "Mango");

        Collections.sort(products, (a, b) -> a.compareTo(b));

        System.out.println(products);  // Output: [Apple, Banana, Mango]
    }
}
```

Reasoning to interviewer:

- compareTo() sorts based on character Unicode order.
- Capital letters come before lowercase by default ("Zebra" < "apple").
- This method is the backbone of natural ordering in TreeSet and TreeMap.

Future scope:

- Handle locale-specific sorting using Collator (important for non-English languages).
- Use List.sort(String::compareTo) for cleaner syntax in Java 8+.

Scenario 45: Case-insensitive sorting using .compareToIgnoreCase()

Now, suppose the list contains ["banana", "Apple", "mango"]. Using .compareTo() would put uppercase "Apple" before lowercase "banana". To sort ignoring case differences, we can use .compareToIgnoreCase().

Answer:

.compareToIgnoreCase() works like .compareTo() but ignores letter casing, making "Apple" and "apple" equivalent for sorting.

Code:

```
import java.util.*;

public class CaseInsensitiveSort {
    public static void main(String[] args) {
        List<String> fruits = Arrays.asList("banana", "Apple", "mango");

        Collections.sort(fruits, (a, b) -> a.compareToIgnoreCase(b));

        System.out.println(fruits);  // Output: [Apple, banana, mango]
    }
}
```

Reasoning to interviewer:

- .compareToIgnoreCase() makes sorting more user-friendly since most users don’t expect "Apple" and "apple" to be separated.
- It avoids issues where uppercase words are sorted before all lowercase words.

Future scope:

- Combine with locale-sensitive collators for natural dictionary ordering.
- Use Comparator.comparing(String::toLowerCase) as an alternative for custom rules.

Scenario 46: Building a large SQL query dynamically using StringBuilder

Suppose you're writing a reporting module where an SQL query needs to be built based on user-selected filters. Concatenating strings repeatedly with + would be inefficient because strings are immutable.

Answer:

StringBuilder is ideal here because it's mutable and efficient for repeated appends in single-threaded contexts.

Code:

```
public class SQLQueryBuilder {
    public static void main(String[] args) {
        StringBuilder query = new StringBuilder("SELECT * FROM users WHERE
1=1");

        // Add filters dynamically
        query.append(" AND age > ").append(25);
        query.append(" AND city = '").append("New York").append("'");
        query.append(" ORDER BY ").append("created_at DESC");

        System.out.println(query.toString());
        // Output: SELECT * FROM users WHERE 1=1 AND age > 25 AND city =
'New York' ORDER BY created_at DESC
    }
}
```

Reasoning to interviewer:

- StringBuilder avoids creating multiple intermediate string objects during concatenation.
- The API (append, insert, delete) is flexible for query building.
- Using it inside loops or conditionals keeps performance optimal compared to string concatenation.

Future scope:

- Use prepared statements (PreparedStatement) for security against SQL injection.
- Wrap query-building in a utility class for maintainability.

Scenario 47: Multi-threaded logging where StringBuffer is safer than StringBuilder

Imagine a logging framework where multiple threads write log messages to a shared buffer. If StringBuilder is used, race conditions can corrupt data.

Answer:

Use StringBuffer because it provides synchronized methods, making it thread-safe.

Code:

```
public class LoggingExample {
    private static StringBuffer logBuffer = new StringBuffer();

    public static void main(String[] args) throws InterruptedException {
        Runnable loggerTask = () -> {
            for (int i = 0; i < 5; i++) {
                logBuffer.append(Thread.currentThread().getName())
                        .append(" - Log entry ").append(i).append("\n");
            }
        };

        Thread t1 = new Thread(loggerTask, "Thread-1");
        Thread t2 = new Thread(loggerTask, "Thread-2");

        t1.start();
        t2.start();
        t1.join();
        t2.join();

        System.out.println(logBuffer.toString());
    }
}
```

Reasoning to interviewer:

- StringBuffer synchronizes its methods, so only one thread modifies the buffer at a time.
- This prevents inconsistent logs when multiple threads append simultaneously.
- The tradeoff is slightly slower performance compared to StringBuilder in single-threaded use.

Future scope:

- For very high-performance logging, use java.util.concurrent structures or libraries like Log4j/SLF4J.
- If only one thread modifies the buffer, prefer StringBuilder for efficiency.

Scenario 48: Reversing a string efficiently with StringBuilder.reverse()

Suppose you're asked to reverse user input "Hello World" into "dlroW olleH". Doing it manually with loops is verbose — but StringBuilder offers a built-in method.

Answer:

StringBuilder.reverse() provides an efficient, in-place reversal of the character sequence.

Code:

```
public class StringReverser {
    public static void main(String[] args) {
        String input = "Hello World";
        String reversed = new StringBuilder(input).reverse().toString();

        System.out.println("Reversed: " + reversed);  // Output: dlroW olleH
    }
}
```

Reasoning to interviewer:

- Creating a StringBuilder around the string allows use of its mutable operations.
- .reverse() is optimized and avoids writing custom logic for reversal.
- Converting back to string with .toString() makes it usable in normal string APIs.

Future scope:

- Combine with palindrome checks (compare input with reversed version).
- Handle Unicode surrogate pairs carefully if processing non-ASCII text.

Scenario 49: Formatting invoice numbers as INV-0001 using String.format()

Suppose you're building a billing system and each invoice number must follow a fixed format like INV-0001, INV-0002, etc. The numeric part should always be padded to 4 digits.

Answer:

We can use String.format() with %04d, which ensures integers are zero-padded to 4 digits.

Code:

```
public class InvoiceFormatter {
    public static void main(String[] args) {
        int invoiceNumber = 1;

        String formatted = String.format("INV-%04d", invoiceNumber);
        System.out.println(formatted);  // Output: INV-0001
    }
}
```

Reasoning to interviewer:

- %04d means: format an integer (d) padded to width 4 with leading zeros (0).
- String.format() makes the template easy to change without complex string concatenation.
- This is commonly used for IDs, invoice numbers, or serials.

Future scope:

- Store the sequence number in a database to avoid duplicates.
- Extend formatting to include year or branch code, e.g., INV-2025-0001.

Scenario 50: Formatting a floating-point number to 2 decimal places

Suppose you're writing an e-commerce checkout page and need to display prices like 12.50 instead of 12.5.

Answer:

We can use String.format() or System.out.printf() with %.2f to round and display exactly two decimal places.

Code:

```
public class PriceFormatter {
    public static void main(String[] args) {
        double price = 12.5;

        String formatted = String.format("%.2f", price);
        System.out.println("Price: $" + formatted);  // Output: Price: $12.50
    }
}
```

Reasoning to interviewer:

- %.2f ensures exactly two decimal places, rounding if needed.
- Works well for financial values where consistent display is important.
- This avoids string concatenation hacks or manual rounding.

Future scope:

- Use DecimalFormat or NumberFormat.getCurrencyInstance() for locale-specific formatting (e.g., ₹12.50 vs $12.50).
- Store raw values as integers (like cents) for precision in databases.

Scenario 51: Formatting date/time into a human-readable string with printf

Suppose you're building a logging system and want to print timestamps in a user-friendly way, like:
2025-09-13 15:30:45

Answer:

You can use printf or String.format() with %t format specifiers for date/time.

Code:

```
import java.util.*;

public class DateTimeFormatterExample {
    public static void main(String[] args) {
        Date now = new Date();

        // yyyy-MM-dd HH:mm:ss
        System.out.printf("%1$tY-%1$tm-%1$td %1$tH:%1$tM:%1$tS%n", now);
        // Example Output: 2025-09-13 15:30:45
    }
}
```

Reasoning to interviewer:

- %tY → year, %tm → month, %td → day.
- %tH:%tM:%tS → hours, minutes, seconds in 24-hour format.
- The 1$ means reuse the first argument (now) for all specifiers.

Future scope:

- For modern Java (8+), use java.time.LocalDateTime and DateTimeFormatter.
- Allow users to select different formats (e.g., MM/dd/yyyy vs dd-MM-yyyy).
- Localize time zones for distributed applications.

Scenario 52: Extracting IP addresses from logs

Suppose your server logs contain entries like:
"User connected from 192.168.0.1 at 10:15 AM"
You need to extract the IP addresses for auditing or analytics.

Answer:

We can use the regex:

\b(?:\d{1,3}\.){3}\d{1,3}\b

- \d{1,3} → matches 1 to 3 digits.
- \. → matches a literal dot.
- (?:...){3} → repeats 3 times for the first three octets.
- Final \d{1,3} captures the last octet.
- \b ensures word boundaries.

Code:

```
import java.util.regex.*;

public class IPExtractor {
    public static void main(String[] args) {
        String log = "User connected from 192.168.0.1 at 10:15 AM, another from
10.0.0.5";
        String regex = "\\b(?:\\d{1,3}\\.){3}\\d{1,3}\\b";

        Pattern pattern = Pattern.compile(regex);
        Matcher matcher = pattern.matcher(log);

        while (matcher.find()) {
            System.out.println("Found IP: " + matcher.group());
        }
    }
}
```

Reasoning to interviewer:

- Regex captures standard IPv4 addresses.
- matcher.find() loops to extract all occurrences.
- Does not validate whether numbers exceed 255, but works for basic log parsing.

Future scope:

- Add stricter validation to disallow invalid IPs like 999.999.999.999.
- Extend regex for IPv6 support.
- Store extracted IPs in a set to avoid duplicates.

Scenario 53: Validating currency amounts like $123.45

Suppose you're processing payment input fields where users type values like $123.45. You need to validate the format.

Answer:

We can use regex:

^\$\d+(\.\d{2})?$

- ^\$ → string must start with $.
- \d+ → one or more digits.
- (\.\d{2})? → optional decimal with exactly two digits.
- $ → end of string.

Code:

```
import java.util.regex.*;

public class CurrencyValidator {
    public static void main(String[] args) {
        String amount = "$123.45";
        String regex = "^\\$\\d+(\\.\\d{2})?$";

        Pattern pattern = Pattern.compile(regex);
        Matcher matcher = pattern.matcher(amount);

        if (matcher.matches()) {
            System.out.println("Valid currency format: " + amount);
        } else {
            System.out.println("Invalid currency format.");
        }
    }
}
```

Reasoning to interviewer:

- Ensures proper structure: a dollar sign, digits, and optional .xx.
- Rejects invalid formats like $12.345 or $abc.
- Good for preliminary validation before converting to BigDecimal.

Future scope:

- Extend to other currency symbols (₹, €, £).
- Use NumberFormat.getCurrencyInstance() for locale-aware formatting.
- Validate range limits (e.g., not exceeding maximum transaction value).

Scenario 54: Extracting HTML tags or attributes

Suppose you're analyzing HTML content and want to extract all tag names or specific attributes (like href).

Answer:

To capture HTML tags:

<([a-zA-Z]+)([^>]*)>

- <([a-zA-Z]+) → captures the tag name.
- ([^>]*) → captures attributes inside the tag.
- > closes the match.

Code:

```
import java.util.regex.*;

public class HTMLTagExtractor {
    public static void main(String[] args) {
        String html = "<a href=\"https://example.com\">Click</a> <div
class='container'>";
        String regex = "<([a-zA-Z]+)([^>]*)>";

        Pattern pattern = Pattern.compile(regex);
        Matcher matcher = pattern.matcher(html);

        while (matcher.find()) {
            System.out.println("Tag: " + matcher.group(1));
            System.out.println("Attributes: " + matcher.group(2).trim());
        }
    }
}
```

Reasoning to interviewer:

- Captures both tag names (a, div) and attributes (href, class).
- Works well for lightweight HTML parsing.
- Not a replacement for full HTML parsers but useful for quick extraction.

Future scope:

- Use libraries like JSoup for production-grade HTML parsing.
- Extend regex to handle self-closing tags (<img />).
- Extract specific attributes (like href) with sub-patterns.

Scenario 55: Checking if a string contains only alphabets

Suppose a signup form requires usernames with only letters, no digits or special characters.

Answer:

We can use regex:

^[A-Za-z]+$

- ^ and $ anchor the match to the entire string.
- [A-Za-z]+ ensures only letters are allowed.

Code:

```
import java.util.regex.*;

public class AlphabetChecker {
    public static void main(String[] args) {
        String input = "Rajat";
        String regex = "^[A-Za-z]+$";

        Pattern pattern = Pattern.compile(regex);
        Matcher matcher = pattern.matcher(input);

        if (matcher.matches()) {
            System.out.println("Valid alphabetic string.");
        } else {
            System.out.println("Invalid string.");
        }
    }
}
```

Reasoning to interviewer:

- Ensures input has only letters, no spaces, numbers, or symbols.
- matcher.matches() enforces full-string validation.
- Good for validating usernames, names, or codes restricted to letters.

Future scope:

- Allow spaces (^[A-Za-z]+$) for full names.
- Use Unicode character classes (\p{L}) for international alphabets.
- Combine with length checks (e.g., minimum 3 characters).

Scenario 56: Password strength validation with regex (uppercase, lowercase, digit, special, length)

Say you're building signup; the rule is: at least 8 chars, must include uppercase, lowercase, a digit, and a special character, and no whitespace.

Answer:

Use lookaheads to assert required character classes, then constrain the overall body.

Code:

```
import java.util.regex.*;

public class PasswordStrength {
    public static void main(String[] args) {
        String[] samples = {
            "Weak1", "Strong@Pass1", "NoDigits!!", "lowercase1!", "UPPER1!",
"Good#Pass9"
        };

        String regex =
            "^(?=\\S{8,})" +          // length ≥ 8 and no whitespace
            "(?=.*[A-Z])" +           // at least one uppercase
            "(?=.*[a-z])" +           // at least one lowercase
            "(?=.*\\d)" +             // at least one digit
            "(?=.*[@$!%*?&#^()_+=\\-])" + // at least one special (customizable set)
            ".*$";

        Pattern p = Pattern.compile(regex);
        for (String s : samples) {
            System.out.printf("%-15s -> %s%n", s, p.matcher(s).matches());
        }
    }
}
```

Reasoning to interviewer:

- Positive lookaheads let me assert multiple requirements without consuming characters.
- \\S{8,} rejects whitespace and enforces minimum length in one shot.
- The special-character set is configurable to your policy.

Future scope:

- Enforce max length, forbid repeating characters, or require two specials.
- Use zxcvbn-like entropy checks for stronger real-world scoring.
- Localize error messages by reporting which rule failed (evaluate lookaheads separately).

Scenario 57: Extracting and strictly validating dates in multiple formats

You receive user dates in yyyy-MM-dd, dd/MM/yyyy, or MM-dd-yyyy. You want both structural and calendar correctness (e.g., reject 2025-02-30).

Answer:

First, use a permissive regex to detect candidate dates; then parse with java.time using strict resolver style to validate calendar rules.

Code:

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.ResolverStyle;
import java.util.List;
import java.util.regex.*;

public class StrictDateValidator {
    private static final List<DateTimeFormatter> FORMATTERS = List.of(
        DateTimeFormatter.ofPattern("uuuu-MM-
dd").withResolverStyle(ResolverStyle.STRICT),

DateTimeFormatter.ofPattern("dd/MM/uuuu").withResolverStyle(ResolverStyle.ST
RICT),
        DateTimeFormatter.ofPattern("MM-dd-
uuuu").withResolverStyle(ResolverStyle.STRICT)
    );

    public static void main(String[] args) {
        String text = "Good: 2025-09-13, 31/01/2025, 12-25-2021. Bad: 2025-02-30,
31/11/2023";
        String candidateRegex = "\\b(\\d{4}-\\d{2}-\\d{2}|\\d{2}/\\d{2}/\\d{4}|\\d{2}-\\d{2}-
\\d{4})\\b";

        Pattern pattern = Pattern.compile(candidateRegex);
        Matcher m = pattern.matcher(text);

        while (m.find()) {
            String token = m.group();
            if (isValidDate(token)) {
                System.out.println("Valid date: " + token);
            } else {
                System.out.println("Invalid date: " + token);
            }
        }
    }

    private static boolean isValidDate(String s) {
```

```
        for (DateTimeFormatter f : FORMATTERS) {
            try {
                LocalDate.parse(s, f);
                return true;
            } catch (Exception ignored) {}
        }
        return false;
    }
}
```

Reasoning to interviewer:

- Regex spots potential dates quickly.
- Strict parsing is the authority for month/day bounds and leap years.
- Separating detection from validation keeps the code clear.

Future scope:

- Capture groups for day/month/year to normalize to ISO after parsing.
- Add locale-driven patterns dynamically.
- Handle times with LocalDateTime and zoned formats if needed.

Scenario 58: Escaping user search terms and combining them in one regex

Users search for literal phrases that may contain regex metacharacters (e.g., C++, price+discount). We want to highlight all matches in one pass.

Answer:

Escape each term with Pattern.quote() and join with |, then do a single scan using appendReplacement.

Code:

```
import java.util.*;
import java.util.regex.*;

public class MultiHighlight {
    public static void main(String[] args) {
        String text = "Learn C++, C#, and price+discount strategies.";
        List<String> terms = List.of("C++", "C#", "price+discount");

        String combined = terms.stream()
                .map(Pattern::quote)
                .reduce((a, b) -> a + "|" + b)
                .orElse("");

        Pattern p = Pattern.compile(combined);
        Matcher m = p.matcher(text);

        StringBuffer sb = new StringBuffer();
        while (m.find()) {
            m.appendReplacement(sb, "[[" + Matcher.quoteReplacement(m.group()) +
"]]");
        }
        m.appendTail(sb);

        System.out.println(sb.toString());
        // Output: Learn [[C++]], [[C#]], and [[price+discount]] strategies.
    }
}
```

Reasoning to interviewer:

- Pattern.quote() treats user input as literal text—no accidental regex behavior.
- One combined pattern = one pass over the text → better performance.

- Matcher.quoteReplacement() keeps the replacement safe when it includes $ or \.

Future scope:

- Add Pattern.CASE_INSENSITIVE and locale-aware folding.
- Pre-compile and cache patterns for repeated queries.
- Return match offsets for UI highlighting.

Scenario 59: String pooling and intern()

You want to show why pooling matters and how intern() can canonicalize equal strings to the same reference.

Answer:

Literals live in the String pool; new String("x") creates a heap object. Calling .intern() returns the pooled reference.

Code:

```
public class StringInternDemo {
    public static void main(String[] args) {
        String a = "Hello";
        String b = "Hello";
        String c = new String("Hello");
        String d = c.intern();

        System.out.println(a == b); // true (same pool entry)
        System.out.println(a == c); // false (heap vs pool)
        System.out.println(a == d); // true (interned matches pool)
        System.out.println(a.equals(c)); // true (content)
    }
}
```

Reasoning to interviewer:

- Pool deduplicates identical literals.
- intern() is useful for memory savings and fast reference comparisons in high-duplication domains (e.g., many repeating keys).

Future scope:

- Be cautious: aggressive interning can increase permgen/metaspace pressure in very old JVMs; modern JVMs handle it better but still consider memory.
- Prefer .equals() for correctness; == only if you fully control interning.

Scenario 60: Advanced formatting with MessageFormat and NumberFormat (i18n currencies)

You need to print localized order confirmations: “Hello, Raj! Your total is ₹1,23,456.79.”

Answer:

Use MessageFormat for templates and NumberFormat.getCurrencyInstance(locale) for currency. This respects locale digits, grouping, and symbols.

Code:

```
import java.text.MessageFormat;
import java.text.NumberFormat;
import java.util.Locale;

public class I18nFormatting {
    public static void main(String[] args) {
        String template = "Hello, {0}! Your total is {1}.";
        double amount = 123456.79;

        Locale india = new Locale("en", "IN");
        NumberFormat inr = NumberFormat.getCurrencyInstance(india);

        String msg = MessageFormat.format(template, "Raj", inr.format(amount));
        System.out.println(msg); // Hello, Raj! Your total is ₹1,23,456.79.
    }
}
```

Reasoning to interviewer:

- MessageFormat cleanly separates text from values.
- NumberFormat handles currency symbol, grouping, and decimals per locale—no manual hacks.

Future scope:

- Externalize templates in resource bundles for full i18n.
- Use DateTimeFormatter for localized dates in the same template.
- Switch locales at runtime based on user preference.

Scenario 61: Extracting the domain from an email using regex groups

You need to store just the domain (e.g., example.com) from emails like user.name+tag@example.com.

Answer:

Capture user and domain in groups (named groups make it clear), then read the domain group.

Code:

```
import java.util.regex.*;

public class EmailDomainExtractor {
    public static void main(String[] args) {
        String email = "user.name+tag@example.com";
        String regex = "^(?<user>[A-Za-z0-9._%+-]+)@(?<domain>[A-Za-z0-9.-
]+\\.[A-Za-z]{2,})$";

        Pattern p = Pattern.compile(regex);
        Matcher m = p.matcher(email);

        if (m.m atches()) {
            String domain = m.group("domain");
            System.out.println("Domain: " + domain); // example.com
        } else {
            System.out.println("Invalid email");
        }
    }
}
```

Reasoning to interviewer:

- Named groups make intent obvious: user, domain.
- The domain pattern tolerates subdomains (a.b.example.co.in).
- Using matches() ensures the whole string is a valid email shape before extracting.

Future scope:

- Additional post-checks (IDN punycode, MX lookup).
- Use InternetAddress for stricter RFC validations when needed.

- Extract organization suffixes (e.g., co.in) using a public suffix list if business logic needs the registrable domain.